# Deep Learning Lab Report

Fiche TP3: Training a CNN From Scratch

ESI-SBA - École Supérieure en Informatique 08-Mai-1945

November 1, 2026

## Contents

**1 Introduction** **2**

**2 Data Pre-processing** **2**
2.1 Data Normalization . . . . . 2
2.2 Data Augmentation . . . . . 2

**3 Core Layer Mechanisms** **2**

**4 Improving the CNN Architecture** **3**
4.1 Batch Normalization . . . . . 3
4.2 Dropout . . . . . 3
4.3 Padding = 'Same' . . . . . 3
4.4 Increased Capacity and Depth . . . . . 3

**5 Performance Metrics Comparison** **3**
5.1 Original CNN Architecture . . . . . 3
5.2 Improved CNN Architecture . . . . . 4

**6 Conclusion** **4**

# 1 Introduction

This report summarizes the concepts, architecture, and mathematical mechanisms involved in building, training, and improving a Convolutional Neural Network (CNN) from scratch in **only 5 epochs** using Keras and TensorFlow. The primary objective of the lab is to classify images from the CIFAR-10 dataset.

# 2 Data Pre-processing

Before feeding data into the neural network, raw image pixels must be pre-processed to improve training efficiency and model generalization.

## 2.1 Data Normalization

Pixel values in the CIFAR-10 dataset range from 0 to 255. We normalize these values to the $[0, 1]$ range by dividing by 255.0. This ensures smaller, more consistent gradient updates, accelerating convergence.

## 2.2 Data Augmentation

To artificially expand the dataset and prevent overfitting, random transformations are applied to the training images during each epoch.

- **Width/Height Shift:** Randomly translating images horizontally or vertically by 10%.
- **Horizontal Flip:** Mirroring images.

This forces the model to learn translation-invariant features rather than memorizing exact spatial locations.

# 3 Core Layer Mechanisms

The initial CNN architecture comprises several distinct layers, each serving a specific purpose.

- **Input Layer:** A placeholder defining the shape of incoming data. For CIFAR-10, the shape is $(32, 32, 3)$ representing $32 \times 32$ pixels across 3 RGB channels.
- **Conv2D (Convolutional Layer):** Extracts spatial features. A sliding filter (kernel) computes dot products over local regions of the input. We applied $3 \times 3$ kernels and the ReLU activation function $f(z) = \max(0, z)$ to introduce non-linearity.
- **MaxPooling2D:** A downsampling operation that selects the maximum value in a given window (e.g., $2 \times 2$). It reduces spatial dimensions, thereby decreasing computational load and introducing local translation invariance.
- **Flatten Layer:** Reshapes the 3D tensor output from the convolutional base into a 1D vector, creating a bridge to the dense layers.

- **Dense (Fully Connected) Layer:** Performs the final classification. Every neuron is connected to every output from the Flatten layer. The final layer uses 10 neurons (for 10 classes) and a *softmax* activation function to output a probability distribution.

# 4 Improving the CNN Architecture

The basic model struggles to hit the target 70% accuracy within 5 epochs. To achieve this, several architectural improvements were implemented.

## 4.1 Batch Normalization

Standardizes the inputs to a layer for each mini-batch.

- **Utility:** Combats internal covariate shift, allowing for higher learning rates and drastically faster convergence.

## 4.2 Dropout

Randomly sets a fraction of input units to 0 during training (e.g., 25% or 50%).

- **Utility:** Acts as strong regularization, preventing neurons from co-adapting and drastically reducing overfitting.

## 4.3 Padding = 'Same'

Adds zero-padding around the image boundaries so the convolution output maintains the same width and height as the input.

- **Utility:** Prevents the rapid loss of spatial dimensions, allowing the network to be built deeper to learn more abstract features.

## 4.4 Increased Capacity and Depth

The filter counts were increased from 16/32 to 32/64, and a hidden Dense layer of 128 neurons was inserted before the output.

- **Utility:** Allows the network to learn a much richer vocabulary of visual features and logically combine them in the dense layer before making a final prediction.

# 5 Performance Metrics Comparison

This section presents the performance metrics (accuracy and loss) evaluated on both the training and test sets over the 5 epochs.

## 5.1 Original CNN Architecture

The original architecture exhibits slower convergence and struggles to reach the required accuracy threshold within the 5-epoch limit.

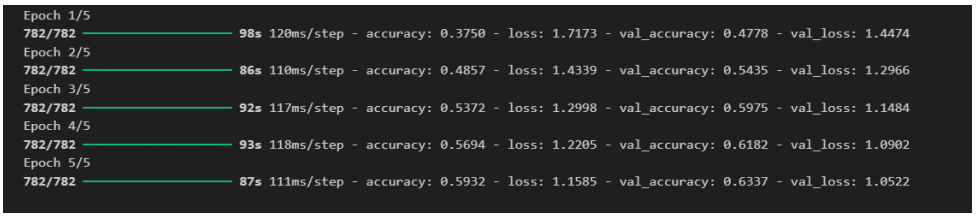



Figure 1: Training and validation metrics for the original CNN architecture.

### 5.2 Improved CNN Architecture

With the addition of Batch Normalization, Dropout, and increased network capacity, the improved model converges significantly faster, successfully achieving the 70% accuracy target on the test set.

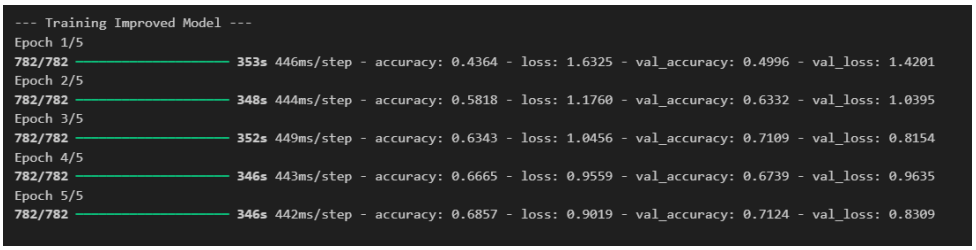



Figure 2: Training and validation metrics for the improved CNN architecture.

## 6 Conclusion

By combining data augmentation to diversify the dataset, and applying modern deep learning techniques such as Batch Normalization and Dropout, a CNN can efficiently learn complex hierarchical features and achieve strong classification accuracy on the CIFAR-10 dataset even within a constrained timeframe of 5 epochs.